

CSE 144 Final Project Report

Transfer Learning Challenge
UC Santa Cruz — Spring 2026

1 Introduction

This project applies transfer learning to a 100-class image classification task. The dataset is small by deep-learning standards — only 10 training images per class — making transfer learning an ideal approach: instead of learning visual features from scratch, we fine-tune models that already possess rich, ImageNet-pretrained representations.

Our approach combines five architectures from different model families (EfficientNet-B0/B3/B4, ResNet-50, and ViT-B/16) into a soft-voting ensemble. Training each model on the full 1,079-image dataset and averaging their softmax outputs yielded a final Kaggle test accuracy of **77.2%**, well above the 60% required baseline.

2 Dataset

The dataset is a 100-class subset sampled from multiple public image datasets. Each class contains approximately 10 training images and 10 evaluation images, giving roughly 1,079 training images in total and 1,036 test images.

2.1 Directory Structure

```
train/  0/  0.jpg ... 9.jpg  1/  ...  99/  
test/   0.jpg ... 1035.jpg
```

2.2 Label Mapping

ImageFolder (PyTorch) sorts class folders alphabetically, causing folder "10" to receive label 2 instead of label 10. We override `class_to_idx` so that folder N always maps to integer label N:

```
full_dataset.class_to_idx = {cls: int(cls) for cls in full_dataset.classes}
```

2.3 Preprocessing & Augmentation

Training augmentations (applied per-epoch to increase effective dataset size):

- Resize to 224×224
- Random horizontal and vertical flip
- Random rotation $\pm 20^\circ$
- Color jitter: brightness, contrast, saturation ± 0.4 ; hue ± 0.1
- Random grayscale ($p=0.10$)

- Normalize with ImageNet mean/std ([0.485, 0.456, 0.406] / [0.229, 0.224, 0.225])

Validation/inference: resize to 224×224, ToTensor, same normalization — no augmentation.

3 Implementation

3.1 Model Architectures

We fine-tuned five pretrained models from TorchVision, spanning two architecture families to maximize ensemble diversity:

Model	Pretrained Weights	Params (M)	Head Replaced
EfficientNet-B0	IMAGENET1K_V1	5.3	classifier[1] → Linear(1280, 100)
EfficientNet-B3	IMAGENET1K_V1	12.2	classifier[1] → Linear(1536, 100)
EfficientNet-B4	IMAGENET1K_V1	19.3	classifier[1] → Linear(1792, 100)
ResNet-50	IMAGENET1K_V2	25.6	fc → Linear(2048, 100)
ViT-B/16	IMAGENET1K_V1	86.6	heads.head → Linear(768, 100)

3.2 Fine-Tuning Strategy

All layers were fine-tuned end-to-end (no frozen layers). With only ~10 images per class, freezing early layers reduced performance in preliminary experiments, while full fine-tuning with strong regularization worked best.

3.3 Training Configuration

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1e-4
Weight decay	1e-2
LR Scheduler	CosineAnnealingLR (T_max = 50)
Loss function	CrossEntropyLoss (label_smoothing = 0.1)
Batch size	32
Epochs	50
Image size	224×224
Random seed	42
Hardware	NVIDIA T4 GPU (Google Colab)
Framework	PyTorch + TorchVision

4 Experiments

4.1 Baseline

The baseline was a single EfficientNet-B0 with basic augmentations (horizontal flip, rotation $\pm 15^\circ$, color jitter), trained on an 80/20 train/val split for 30 epochs. This achieved 58.6% validation accuracy and 65.4% Kaggle test accuracy.

4.2 Ablation Studies

Experiment	Val Acc	Kaggle Acc	Change
B0 baseline (30 epochs, weak aug)	58.6%	65.4%	—
B0 + stronger aug + label smoothing	62.3%	65.4%	baseline
B0 + B3 ensemble (80% train data)	64.7%	66.3%	+0.9%
B0 + B3 + R50 (full dataset)	—	74.5%	+8.2%
B0+B3+B4+R50+ViT (full dataset)	—	77.2%	+2.7%
5-model + TTA	—	75.0%	-2.2%

4.3 Key Design Decisions

- **Full dataset training:** Removing the validation holdout and training on all 1,079 images gave the largest single improvement (+8.2%), confirming that data is the bottleneck.
- **Ensemble diversity:** Combining CNN-family (EfficientNet, ResNet) and attention-based (ViT) models improved accuracy because their errors are less correlated.
- **Label smoothing:** Prevented the model from becoming overconfident, which is critical when training accuracy reaches 100% well before the dataset is fully captured.
- **TTA was harmful:** Test-time augmentation (flips, rotations, center crop) decreased accuracy by 2.2%, suggesting some test classes are orientation-sensitive.

5 Results

The final submission used a 5-model ensemble (EfficientNet-B0, B3, B4, ResNet-50, ViT-B/16), each trained on the full training set for 50 epochs. Predictions were generated by averaging softmax probabilities across all five models.

Final Kaggle public leaderboard score: 0.772 (77.2%)

Kaggle score (S_{kaggle}) = $70 + \max(0, 77.2 - 60) = \mathbf{87.2 / 100}$

During model selection (80/20 split), the best single-model validation accuracy was 62.3% (EfficientNet-B0), while the 2-model ensemble achieved 64.7% on the same split, confirming that ensembling consistently improved performance.

6 Discussion

6.1 What Worked Best

- Training on the full dataset (largest single gain: +8.2%)
- Diverse ensemble across architecture families (CNN vs. Transformer)
- Label smoothing + high weight decay to control overfitting
- Strong data augmentation (vertical flip, color jitter, grayscale)

6.2 Failure Cases & Observations

All models saturated training accuracy at ~100% by epoch 20-30, while validation accuracy plateaued around 62%, indicating significant overfitting. This is expected given only 10 training images per class. TTA hurt performance, likely because some classes (e.g. text, digits, oriented objects) are not rotationally symmetric.

6.3 Limitations & Future Work

- More models in the ensemble (DenseNet, EfficientNet-B5) would likely help further
- Semi-supervised learning using unlabeled test images could leverage more data
- Per-class TTA masking (skip TTA for orientation-sensitive classes) could recover the TTA loss
- Mixup or CutMix augmentation may further reduce overfitting

7 Reproducibility

7.1 Random Seeds

```
SEED = 42
random.seed(SEED); np.random.seed(SEED)
torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
```

7.2 Environment

Package	Version
Python	3.10
PyTorch	2.x (Colab default)
TorchVision	0.x (Colab default)
NumPy	1.x
Pandas	2.x
Hardware	NVIDIA T4 GPU (Google Colab)

7.3 Running the Code

Training:

1. Open CSE144_Final_Project_Clean.ipynb in Google Colab
2. Set Runtime → T4 GPU
3. Run all cells top to bottom
4. Model weights saved automatically to Google Drive

Inference (generate submission.csv):

1. Run Section 7 (Load Trained Models)
2. Run Section 8 (Generate Submission)
3. Output: /content/drive/MyDrive/CSE144_Project/submission_5model.csv

8 Team Contributions

- Member 1: Data preprocessing, label mapping fix, training pipeline
- Member 2: Model selection, hyperparameter tuning, ensemble design
- Member 3: Experiments, ablation studies, report writing

9 References

- TorchVision Models — <https://pytorch.org/vision/stable/models.html>
- Tan & Le (2019). EfficientNet: Rethinking Model Scaling for CNNs. ICML 2019.
- He et al. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.
- Dosovitskiy et al. (2020). An Image is Worth 16x16 Words. ICLR 2021.
- Loshchilov & Hutter (2019). Decoupled Weight Decay Regularization. ICLR 2019.